

To complete **Stage 2: insert into database**, you need to update your `POST /tasks` endpoint. Instead of checking a local array length and pushing data to it, you will use a parameterized `INSERT` query. This allows SQLite to handle ID incrementing automatically.

Choose the code block below that matches your project lane.

## ● Node.js Lane (Express + better-sqlite3)

Update the creation endpoint in your `server.js` file. `better-sqlite3` provides a `.lastInsertRowid` property on the object returned by `.run()`. You can use this property to fetch the newly created ID.

### javascript

```
// =====  
// Stage 2: POST /tasks (Insert Task)  
// =====  
app.post('/tasks', (req, res) => {  
  const { title } = req.body;  
  
  // Validation: Missing or empty title returns 400 Bad Request  
  if (!title || title.trim() === "") {  
    return res.status(400).json({ error: "Title is required" });  
  }  
  
  // Insert into database; pass title cleanly and set done to 0 (false)  
  const insertStmt = db.prepare('INSERT INTO tasks (title, done) VALUES (?, 0)');  
  const result = insertStmt.run(title);  
  
  // Retrieve the autogenerated ID assigned by SQLite  
  const newId = result.lastInsertRowid;  
  
  // Respond with 201 Created and the fresh record object  
  res.status(201).json({  
    id: Number(newId),  
    title: title,  
    done: false  
  });  
});
```

Use code with caution.

## ● Python Lane (FastAPI + sqlite3)

Update your `POST` endpoint logic in your `main.py` file. Python's `sqlite3` cursor exposes a `.lastrowid` attribute after executing an `INSERT` statement. Make sure to call `conn.commit()` so your structural changes write permanently to disk.

python

```
# =====
# Stage 2: POST /tasks (Insert Task)
# =====
@app.post("/tasks", status_code=status.HTTP_201_CREATED)
def create_task(task_input: TaskCreate):
    # Validation: Empty string inputs return a 400 Bad Request
    if not task_input.title.strip():
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail={"error": "Title cannot be empty"}
        )

    conn = get_db_connection()
    cursor = conn.cursor()

    # Run parameterized insert statement
    cursor.execute(
        "INSERT INTO tasks (title, done) VALUES (?, 0)",
        (task_input.title,)
    )

    # Commit changes permanently to the tasks.db file
    conn.commit()

    # Extract the auto-incremented primary key assigned by SQLite
    new_id = cursor.lastrowid
    conn.close()

    return {
        "id": new_id,
        "title": task_input.title,
```

```
"done": false  
}
```

Use code with caution.

## ❏ Checkpoint Verification

Verify your server's data persistence properties using your platform terminal tool:

### 1. Send creation requests to populate records:

bash

```
curl -i -X POST -H "Content-Type: application/json" -d '{"title":"Mow the lawn"}' http  
curl -i -X POST -H "Content-Type: application/json" -d '{"title":"Fix the sink"}' http
```

Use code with caution.

2. **Kill your server process entirely:** Go to your terminal running the application process and press `Ctrl + C`.

3. **Boot your server up again** and run a standard fetch request:

bash

```
curl -i http://localhost:3000/tasks
```

Use code with caution.

*Expected Outcome:* The newly added database values appear intact in the return list payload, surviving the server crash or restart sequence completely!

## Git Commit

Commit your Stage 2 changes to your version history tree:

bash

```
git add server.js main.py  
git commit -m "Stage 2: insert into database"  
git push
```